

Week 4 - Feature Engineering

Contents

Goal	1
1. Setup	1
2. Load the Sea Surface Temperature Dataset	2
3. Subset the Data	2
4. Explore the Data Visually	3
5. Convert Gridded Data to a Machine-Learning Table	4
6. Define the Prediction Problem	5
7. Create a Reusable Model and Evaluation Function	6
8. Naive Evaluation: Random k-Fold Cross-Validation	7
9. Feature Engineering	7

Goal

Build a non-leaking feature for a gridded environmental dataset and compare model evaluation strategies.

In this activity, you will use a gridded **sea surface temperature (SST)** dataset from oceanography. SST varies smoothly across space and seasonally through time, so nearby observations are not independent.

Dataset: NOAA Extended Reconstructed Sea Surface Temperature version 5. NOAA describes ERSSTv5 as a global monthly SST analysis on a 2-degree grid derived from ICOADS, with records extending from 1854 to the present.

Useful dataset references:

- <https://www.psl.noaa.gov/data/gridded/data.noaa.ersst.v5.html>
- <https://downloads.psl.noaa.gov/Datasets/noaa.ersst.v5/sst.mnmean.nc>

1. Setup

Run the chunk below. If any package is missing, uncomment the install line and run it once.

This R Markdown document intentionally gives you some complete code and some incomplete code. Sections marked **TODO** are places where you should write or modify code yourself.

```
# Uncomment if needed:
# install.packages(c("tidymodels", "terra", "lubridate", "ranger"))

library(tidymodels)
library(terra)
library(lubridate)

theme_set(theme_minimal())

set.seed(42)
```

2. Load the Sea Surface Temperature Dataset

The data are stored as a NetCDF file, which is common for gridded environmental and oceanographic data. Instead of rows and columns only, the data have dimensions such as time, lat, and lon.

The SST variable is measured over a regular latitude-longitude grid through time.

```
sst_url <- "https://downloads.psl.noaa.gov/Datasets/noaa.ersst.v5/sst.mnmean.nc"
sst_file <- "sst.mnmean.nc"
```

```
download.file(sst_url, sst_file, mode = "wb")
```

```
sst_all <- rast(sst_file, subds = "sst")
sst_all
```

```
## class      : SpatRaster
## size       : 89, 180, 2072  (nrow, ncol, nlyr)
## dimensions  : lon, lat, time (180, 89, 2072)
## resolution  : 2, 2  (x, y)
## extent     : -1, 359, -89, 89  (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (CRS84) (OGC:CRS84)
## source      : sst.mnmean.nc
## varname     : sst (Monthly Means of Sea Surface Temperature)
## names       : sst_1, sst_2, sst_3, sst_4, sst_5, sst_6, ...
## unit        : degC
## time (days) : 1854-01-01 to 2026-08-01 (2072 steps)
```

3. Subset the Data

The full global dataset is larger than we need for this activity. The chunk below selects a broad Pacific Ocean region and thins the grid and time axis so the model runs quickly.

We are still keeping enough data to preserve spatial and temporal correlation.

```
sst_dates <- time(sst_all)
```

```
keep_time <- which(
  sst_dates >= as.Date("1992-01-01") &
  sst_dates <= as.Date("2020-12-31")
)
```

```
# Keep every third month.
```

```
keep_time <- keep_time[seq(1, length(keep_time), by = 3)]
```

```
sst_sub <- sst_all[[keep_time]]
```

```
# Select a broad Pacific region.
```

```
# Longitude is stored from 0 to 360 degrees in this dataset.
```

```
sst_sub <- crop(sst_sub, ext(120, 290, -60, 60))
```

```
# Thin the grid by aggregating neighboring cells.
```

```
sst_sub <- aggregate(sst_sub, fact = 2, fun = mean, na.rm = TRUE)
```

```
dates_sub <- sst_dates[keep_time]
```

```
names(sst_sub) <- paste0("t", seq_along(dates_sub))
```

```
sst_sub
```

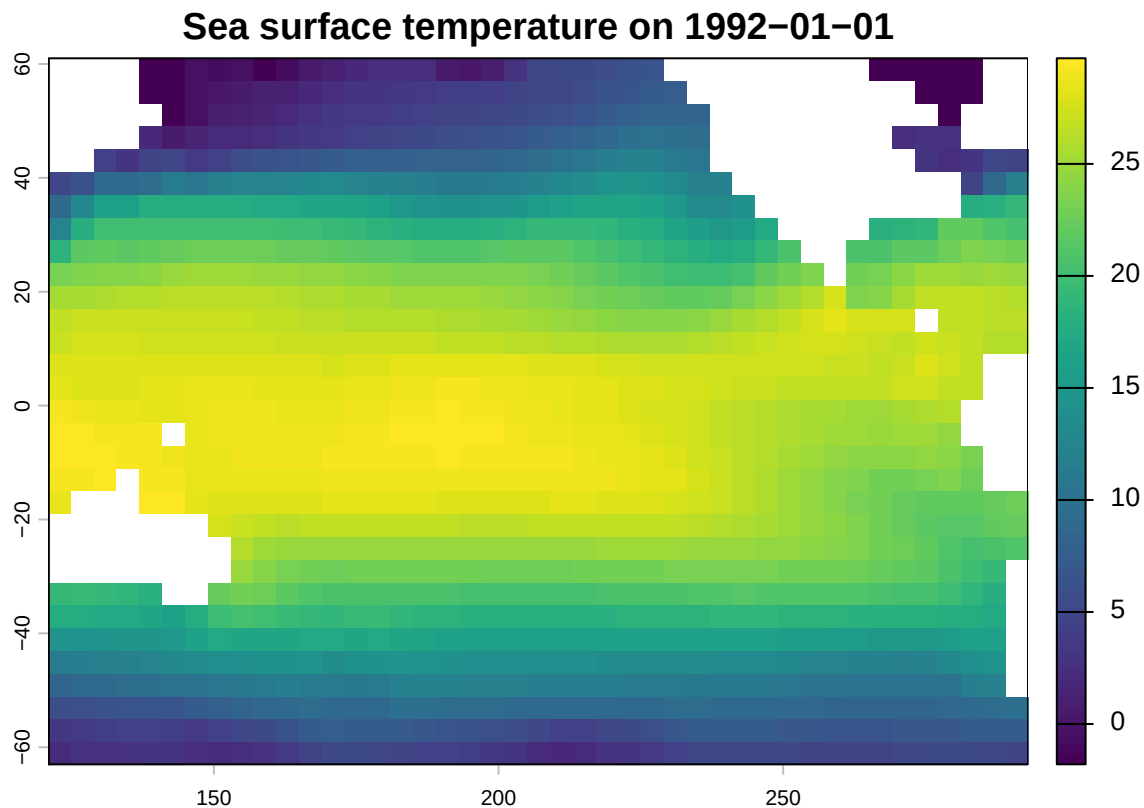
```
## class      : SpatRaster
## size       : 31, 43, 116 (nrow, ncol, nlyr)
## resolution : 4, 4 (x, y)
## extent     : 121, 293, -63, 61 (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (CRS84) (OGC:CRS84)
## source(s)  : memory
## names      :      t1,      t2,      t3,      t4,      t5,      t6, ...
## min values  :     -1.8,     -1.8,     -1.8, -0.908347,     -1.8,     -1.8, ...
## max values  : 29.714429, 29.979055, 30.337552, 30.435486, 29.903572, 29.638452, ...
## time (days) : 1992-01-01 to 2020-10-01 (116 steps)
```

4. Explore the Data Visually

Before modeling, inspect the target variable. SST has strong geographic structure and seasonal/temporal structure.

The first plot shows the SST field for one time step. Nearby ocean cells usually have similar temperatures, which is spatial autocorrelation.

```
plot(sst_sub[[1]], main = paste("Sea surface temperature on", dates_sub[1]))
```

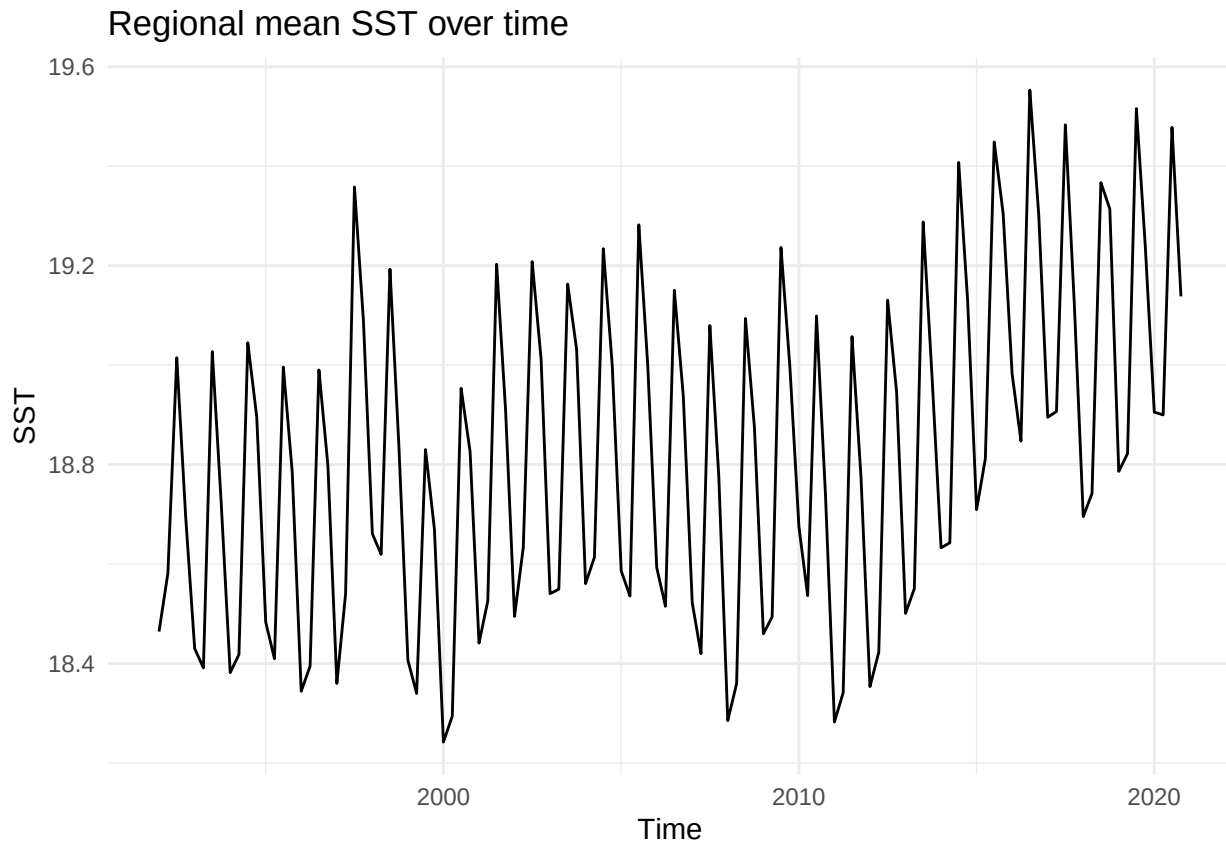


Explore Change Through Time

The next plot averages SST across the selected region for each time step. Because we kept every third month, the pattern will still show seasonal and long-term variation.

```
regional_mean <- global(sst_sub, fun = mean, na.rm = TRUE) %>%
  as_tibble() %>%
  mutate(time = dates_sub)
```

```
ggplot(regional_mean, aes(x = time, y = mean)) +
  geom_line() +
  labs(
    title = "Regional mean SST over time",
    x = "Time",
    y = "SST"
  )
```



5. Convert Gridded Data to a Machine-Learning Table

Most machine-learning models expect a table where each row is one observation. Here, one row represents one grid cell at one time.

The target is `sst`. The predictors will be simple features based on location and time.

We create cyclic features for month because December is close to January.

```
df_wide <- as.data.frame(sst_sub, xy = TRUE, na.rm = TRUE)

date_lookup <- tibble(
  layer = names(sst_sub),
  time = dates_sub
)

df <- df_wide %>%
  pivot_longer(
    cols = starts_with("t"),
    names_to = "layer",
```

```

    values_to = "sst"
  ) %>%
  left_join(date_lookup, by = "layer") %>%
  rename(lon = x, lat = y) %>%
  mutate(
    year = year(time),
    month = month(time)
  )

# TODO: create month_sin and month_cos features.
# Hint: use sin(2 * pi * month / 12) and cos(2 * pi * month / 12).

df <- df %>%
  mutate(
    month_sin = sin(2 * pi * month / 12), # TODO: replace with month sine feature
    month_cos = cos(2 * pi * month / 12) # TODO: replace with month cosine feature
  )
# month_sin has big numbers in the NH Summer
# month_cos has big numbers in the NH Winter
# I usually list if statement/for loop assigning "Winter", "Spring", "Summer", "Fall" and count December a

dim(df)

## [1] 136996      9

head(df)

```

```

## # A tibble: 6 x 9
##   lon  lat layer  sst time      year month month_sin month_cos
##   <dbl> <dbl> <chr> <dbl> <date>    <dbl> <dbl>    <dbl>    <dbl>
## 1  139   59 t1    -1.80 1992-01-01 1992     1      0.5      0.866
## 2  139   59 t2    -1.80 1992-04-01 1992     4      0.866    -0.5
## 3  139   59 t3     9.27 1992-07-01 1992     7     -0.50    -0.866
## 4  139   59 t4     5.21 1992-10-01 1992    10    -0.866     0.5
## 5  139   59 t5    -1.80 1993-01-01 1993     1      0.5      0.866
## 6  139   59 t6    -1.80 1993-04-01 1993     4      0.866    -0.5

```

6. Define the Prediction Problem

We will predict SST from simple location and time features.

This is not meant to be the best possible ocean forecasting model. The goal is to study how evaluation changes when data are spatially and temporally correlated.

```

feature_cols <- c(
  "lat",
  "lon",
  "month_sin",
  "month_cos",
  "year"
)

target_col <- "sst"

model_data <- df %>%
  select(all_of(feature_cols), all_of(target_col))

```

```
head(model_data)
```

```
## # A tibble: 6 x 6
##   lat lon month_sin month_cos year sst
##   <dbl> <dbl>   <dbl>   <dbl> <dbl> <dbl>
## 1   59  139     0.5     0.866  1992 -1.80
## 2   59  139     0.866   -0.5    1992 -1.80
## 3   59  139    -0.50   -0.866  1992  9.27
## 4   59  139    -0.866    0.5    1992  5.21
## 5   59  139     0.5     0.866  1993 -1.80
## 6   59  139     0.866   -0.5    1993 -1.80
```

7. Create a Reusable Model and Evaluation Function

We use a tree-based regression model through `tidymodels`. The `ranger` engine fits a random forest. It can capture nonlinear patterns such as warmer temperatures near the equator and seasonal variation.

The helper function below runs resampling and returns RMSE, MAE, and traditional R-squared.

Interpretation of metrics:

- **RMSE:** Typical prediction error, with larger errors penalized more strongly.
- **MAE:** Typical absolute error.
- **R-squared:** Fraction of variance explained. Higher is better, but it can be misleading when validation folds are not independent.

```
regression_metrics <- metric_set(rmse, mae, rsq_trad)

make_workflow <- function(features) {
  formula_text <- paste("sst ~", paste(features, collapse = " + "))

  sst_recipe <- recipe(as.formula(formula_text), data = model_data) %>%
    step_impute_median(all_numeric_predictors())

  rf_spec <- rand_forest(
    trees = 200,
    min_n = 5
  ) %>%
    set_engine("ranger") %>%
    set_mode("regression")

  workflow() %>%
    add_recipe(sst_recipe) %>%
    add_model(rf_spec)
}

summarize_resamples <- function(results, label) {
  collect_metrics(results) %>%
    filter(.metric %in% c("rmse", "mae", "rsq_trad")) %>%
    mutate(Evaluation = label) %>%
    select(Evaluation, .metric, mean, std_err)
}
```

8. Naive Evaluation: Random k-Fold Cross-Validation

Run the chunk below and record the results.

Random k-fold cross-validation mixes nearby places and nearby times across folds. That can make performance look better than it would be when predicting genuinely new regions or future periods.

```
set.seed(42)

random_folds <- vfold_cv(model_data, v = 5)

random_results <- fit_resamples(
  make_workflow(feature_cols),
  resamples = random_folds,
  metrics = regression_metrics
)

random_summary <- summarize_resamples(random_results, "Random k-fold CV")
random_summary

## # A tibble: 3 x 4
##   Evaluation      .metric  mean  std_err
##   <chr>          <chr>    <dbl>   <dbl>
## 1 Random k-fold CV mae      0.968 0.0100
## 2 Random k-fold CV rmse     1.20  0.0113
## 3 Random k-fold CV rsq_trad 0.982 0.000337
```

9. Feature Engineering

TODO: Add a historic monthly average temperature for each location.

Be careful not to use future data when creating this column. For each grid cell and month, the value should be based only on previous observations from the same grid cell and calendar month.

```
Month_Average_sst=NULL

for (i in 1:length(df$lon)){
  sample_row=df[i,]
  df_subset=df%>%
    filter(lon==sample_row$lon &
           lat==sample_row$lat &
           month==sample_row$month &
           year<sample_row$year) # Lydia's grossest way to do it

  Month_Average_sst[i]=mean(df_subset$sst)
}

df$Month_Average_sst=Month_Average_sst # TODO: replace with historical monthly average

df %>%
  select(lat, lon, time, month, sst, Month_Average_sst) %>%
  arrange(lat, lon, month, time) %>%
  head(15)

## # A tibble: 15 x 6
```

	lat	lon	time	month	sst	Month_Average_sst
	<dbl>	<dbl>	<date>	<dbl>	<dbl>	<dbl>
## 1	-61	123	1992-01-01	1	2.26	NaN
## 2	-61	123	1993-01-01	1	2.48	2.26
## 3	-61	123	1994-01-01	1	1.72	2.37
## 4	-61	123	1995-01-01	1	2.08	2.15
## 5	-61	123	1996-01-01	1	1.88	2.14
## 6	-61	123	1997-01-01	1	3.02	2.08
## 7	-61	123	1998-01-01	1	2.35	2.24
## 8	-61	123	1999-01-01	1	2.34	2.25
## 9	-61	123	2000-01-01	1	1.37	2.27
## 10	-61	123	2001-01-01	1	2.77	2.17
## 11	-61	123	2002-01-01	1	2.30	2.23
## 12	-61	123	2003-01-01	1	2.71	2.23
## 13	-61	123	2004-01-01	1	2.50	2.27
## 14	-61	123	2005-01-01	1	2.75	2.29
## 15	-61	123	2006-01-01	1	2.98	2.32

Hint: one safe pattern is to combine `cummean()` with `lag()` so the current row is not included in its own historical average.

```
feature_cols <- c(
  "lat",
  "lon",
  "month_cos",
  "month_sin",
  "year",
  "Month_Average_sst"
)

model_data <- df %>%
  select(all_of(feature_cols), all_of(target_col))

head(model_data)
```

```
## # A tibble: 6 x 7
##   lat lon month_cos month_sin year Month_Average_sst sst
##   <dbl> <dbl>   <dbl>   <dbl> <dbl>   <dbl> <dbl>
## 1  59  139    0.866    0.5   1992     NaN  -1.80
## 2  59  139   -0.5    0.866  1992     NaN  -1.80
## 3  59  139  -0.866   -0.50  1992     NaN   9.27
## 4  59  139    0.5   -0.866  1992     NaN   5.21
## 5  59  139    0.866    0.5   1993    -1.80 -1.80
## 6  59  139   -0.5    0.866  1993    -1.80 -1.80
```

```
set.seed(42)

random_folds <- vfold_cv(model_data, v = 5)

random_results_with_feature <- fit_resamples(
  make_workflow(feature_cols),
  resamples = random_folds,
  metrics = regression_metrics
)

random_summary_with_feature <- summarize_resamples(
```

```

random_results_with_feature,
  "Random k-fold CV with historical monthly mean"
)

```

```

random_summary_with_feature

```

```

## # A tibble: 3 x 4
##   Evaluation .metric mean std_err
##   <chr>      <chr>   <dbl>   <dbl>
## 1 Random k-fold CV with historical monthly mean mae      0.449 0.00358
## 2 Random k-fold CV with historical monthly mean rmse      0.680 0.00645
## 3 Random k-fold CV with historical monthly mean rsq_trad 0.994 0.000110

```